

Confluent Kafka Isotope

`confluent-kafka-isotope` is a reference implementation of an e-commerce order pipeline that uses Kafka Interceptors with Apache Flink to capture and report event tracing data — both in batch and near real time.

Much like isotopes traced through a biochemical pathway, each event carries metadata that allows it to be tracked as it moves through Kafka topics and distributed microservices.

Kafka topics become the connective tissue between services, while Kafka Interceptors quietly transform the pipeline itself into an observable distributed system.

Table of Contents

- **1.0 How the isotope is carried**
 - **1.1 How the producer interceptor gets invoked**
 - **1.2 Using the Kafka client interceptor vs explicit library calls**
 - **1.3 Why explicit calls for consume-side markers?**
 - **1.4 Why "bipartite"?**
 - **2.0 Architecture**
 - **3.0 Repo layout**
 - **4.0 Running**
 - **4.1. Unit tests (no broker, instant)**
 - **4.2 Demo CLI — see one trace propagate live**
 - **4.2.1 Full Bipartite Demo**
 - **4.3 Integration tests (live Kafka via Minikube)**
 - **4.4 Flink SQL reports on Confluent Platform for Apache Flink (Minikube)**
 - **4.5 Flink SQL reports on Confluent Cloud for Apache Flink (CCAF)**
 - **4.6 Stateless reports via Micrometer → Prometheus/Grafana (optional)**
 - **4.6.1 Enabling the exporter**
 - **4.6.2 The three reports as PromQL**
 - **4.6.3 Consume-side meters**
 - **4.6.4 What stays in Flink — and two deliberate gaps**
 - **4.6.5 One-command showcase: Prometheus + Grafana on Minikube**
 - **5.0 Resources**
-

1.0 How the isotope is carried

An isotope is a lightweight tracing artifact attached to Kafka record headers. Like a biochemical isotope used to trace molecules through a metabolic pathway, it allows the journey of a record through an event-driven architecture to be observed and analyzed.

Like many software patterns, Isotope Tracing can be expressed in mathematical terms. This project renders a Kafka pipeline as a **bipartite graph** — services on one vertex set, topics on the other, with edges running both ways between them — so every produce edge, consume edge, and terminal consumer becomes a first-class citizen of a single topology view.

The tracer that makes this possible is the isotope carried in each record header and observed at two points. On the produce side, a Kafka producer interceptor stamps the isotope and appends one hop per `send()` call, creating the graph's produce edges in `hops[]`. On the consume side, `IsotopeContext.recordConsume(record, service, producer)` forwards the isotope to a value-less marker on `isotope_consume_edge_markers`, creating the consume edges. Services that consume and then produce call `IsotopeContext.adoptFromRecord(record)` in between so the trace identity survives each hop.

Apache Flink reads only the headers and reconstructs what the isotopes reveal: **end-to-end latency**, the **complete service→topic→service topology**, **drop and duplication rates**, and **pipeline coverage**.

Viewed through the biochemical analogy, Kafka records are the molecules moving through the system, while isotopes are the labels that make those movements observable. The goal is not to monitor the isotopes themselves, but to understand the behavior of the event-driven architecture they traverse. By following isotopes through topics and services, Apache Flink can reconstruct the pathways records take, measure how long they spend at each stage, detect where they are dropped or duplicated, and reveal the topology of the system as it actually operates.

A point worth affirming up front: **open-source Apache Flink and Confluent Cloud for Apache Flink (CCAF) run this the same way**. The identical isotope mechanism works against both **Confluent Platform for Apache Flink** (self-managed, the open-source Apache Flink runtime) and **CCAF** (managed) — both driven here via Flink SQL plus an uploaded PTF JAR, no `DataStream` code on either side. Three design choices keep it that way: tagging happens in a Kafka **producer interceptor** (the one extension point both runtimes share via the broker), the on-wire **header** format is **JSON** (so Flink SQL can read the scalar fields with `CAST(headers [...] AS STRING)` and no UDF), and the two stateful reports (`LatencyPercentilesPTF`, `StuckTracePTF`) ship as a single JAR that registers identically on either runtime. Both are **ProcessTableFunctions** — even percentiles, which would naturally be an aggregate function, because CCAF does not support user-defined aggregates currently.

One asymmetry the runtimes don't share: **Flink-native SR-Protobuf**. CCAF supports SR-framed Protobuf as a Flink sink format via its topic catalog; Apache Flink open-source (the CP Flink runtime) ships **avro-confluent** but no SR-Protobuf counterpart. So Flink *report sinks* land on **Avro+SR on CP** and can be **Protobuf+SR on CCAF** — a runtime constraint, not a project preference. The demo *event* topics (next paragraph) are unaffected because they're written by the Kafka producer client, not by Flink.

Message **values** on the demo topics are **SR-framed Protobuf** (`ai.signalroom.kafka.isotope.proto.DemoEvent`) — the standard Confluent value format. The interceptors and reports are agnostic to value format because the isotope rides in headers; the Protobuf choice just gives the integration tests and the demo CLI a typed payload to work with. The **headers** are where the isotope lives, and they have two parts:

- **Header x-isotope** (JSON bytes) carries the full hop history, forwarded by every hop:
 - `t` — 16-byte **UUIDv7** trace ID ([RFC 9562](#)): 48-bit ms timestamp in the high bits + 74 bits random. Stable for the life of the trace, and lexicographic byte order matches creation order — sort trace IDs and you get chronological order for free. The random bits come from `ThreadLocalRandom`, not `SecureRandom`: a trace ID is a public observability identifier carried in Kafka headers, so the requirement is collision avoidance, not unpredictability — and `ThreadLocalRandom` delivers that without `SecureRandom`'s per-call cost on every produced record.

- **o** — origin timestamp (ms) — same value as the timestamp embedded in the UUIDv7 trace ID; kept as its own field for typed access from Flink SQL without needing to decode the UUID bytes.
- **s** — origin service name (set once, never reassigned)
- **p** — origin pipeline name (e.g. **orders** vs **location**); like **s**, stamped once at the origin and forwarded unchanged on every hop, so reports can slice traces by which logical pipeline they belong to
- **h** — ordered list of hops, each `{s: service, t: topic, m: tsMs}`
- **x** — **true** if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted
- **Seven scalar headers** (UTF-8 strings) carry the most-recent-hop view so Flink SQL can read them via `CAST(headers['x-isotope-...'] AS STRING)` without parsing the JSON array (no UDF required on either CCAF or CP Flink). See [scripts/flink/README.md](#) for the full header table.

Example of the Isotope's two parts (formatted for readability; the actual header is in JSON bytes):

```
{
  "x-isotope-hop-count": "1",
  "x-isotope": "
{\\\"t\\\":\\\"AZ690S80eG+9zufGfF2sbw==\\\",\\\"o\\\":1781291101966,\\\"s\\\":\\\"order-intake-
service\\\",\\\"p\\\":\\\"order\\\",\\\"h\\\": [{\\\"s\\\":\\\"order-intake-
service\\\",\\\"t\\\":\\\"orders.placed\\\",\\\"m\\\":1781291101966}],\\\"x\\\":false}\",
  "x-isotope-trace-id": "019ebd392f0e786fbdcee7c67c5dac6f",
  "x-isotope-this-service": "order-intake-service",
  "x-isotope-origin-service": "order-intake-service",
  "x-isotope-this-topic": "orders.placed",
  "x-isotope-origin-ts": "1781291101966",
  "x-isotope-pipeline": "order"
}
```

A producer with the isotope interceptor loaded appends one hop on every `send()`. A consume-then-produce service calls `IsotopeContext.adoptFromRecord(record)` between consume and produce so the trace ID and origin survive the hop.

1.1 How the producer interceptor gets invoked

Application code never calls `onSend` / `onAcknowledgement` directly — the Kafka client invokes them by reflection once two things are in place:

1. **Register the class in producer config.** Put `IsotopeProducerInterceptor.class.getName()` under `interceptor.classes` ([App.java:199](#), [App.java:284](#), [IsotopeTestHarness.java:96](#)). The `KafkaProducer` constructor instantiates the interceptor and owns its lifecycle.
2. **Call `producer.send(...)` as normal.** Every `send()` triggers `onSend` (caller thread, before serialization) and later `onAcknowledgement` (producer I/O thread, after broker ack or failure).

The exact call sites in `kafka-clients` 4.2.0: `KafkaProducer.send` invokes `interceptors.onSend` ([line 950](#)) and `AppendCallbacks.onCompletion` invokes `interceptors.onAcknowledgement` ([line 1600](#)). Exceptions thrown by the interceptor are caught and logged by the client's fan-out wrapper — they never break the `send` call.

1.2 Using the Kafka client interceptor vs explicit library calls

This project uses one Kafka client interceptor, and it's a **ProducerInterceptor** — not a **ConsumerInterceptor**. A **ConsumerInterceptor.onConsume** sees a whole batch from **poll()**, but the application processes records one at a time, so a thread-local snapshot from the batch would be ambiguous about which record is being handled. An earlier **IsotopeConsumerInterceptor** was tried and removed for exactly that reason (see #30). The consume side instead runs on **explicit per-record library calls**: services call **IsotopeContext.adoptFromRecord(record)** between consume and produce to carry the trace through a hop (the next **send()** then sees that thread-local and appends a new hop), and **IsotopeContext.recordConsume(record, service, markerProducer)** to emit a consume-edge marker (see the next paragraph). So: **producer interceptor for produce-side stamping; explicit per-record calls for everything consume-side**.

1.3 Why explicit calls for consume-side markers?

Consume-side markers — the other half of the bipartite graph. The produce-side **hops[]** chain captures **producer → topic** edges and implicitly captures consume edges for *intermediate* services (svc-B consuming from topic-AB is implied by svc-B's next produced hop), but **terminal consumers** — services that consume but never produce — would otherwise be invisible. The bipartite story closes that gap with one library call: consumers call **IsotopeContext.recordConsume(record, service, markerProducer)** between consume and process, which forwards the inbound record's seven scalar **x-isotope-*** headers to a value-less marker record on **isotope_consume_edge_markers** and adds one new header — **x-isotope-consumer-service** — that names the consumer. The **bipartite_topology** Flink report unions these markers with the produce-side view to emit edges in both directions: **producer → topic** (from **isotope**) and **topic → consumer** (from **consume_events**). Markers are fire-and-forget: a dropped marker leaves a hole in the topology graph but never disrupts the consume/produce pipeline.

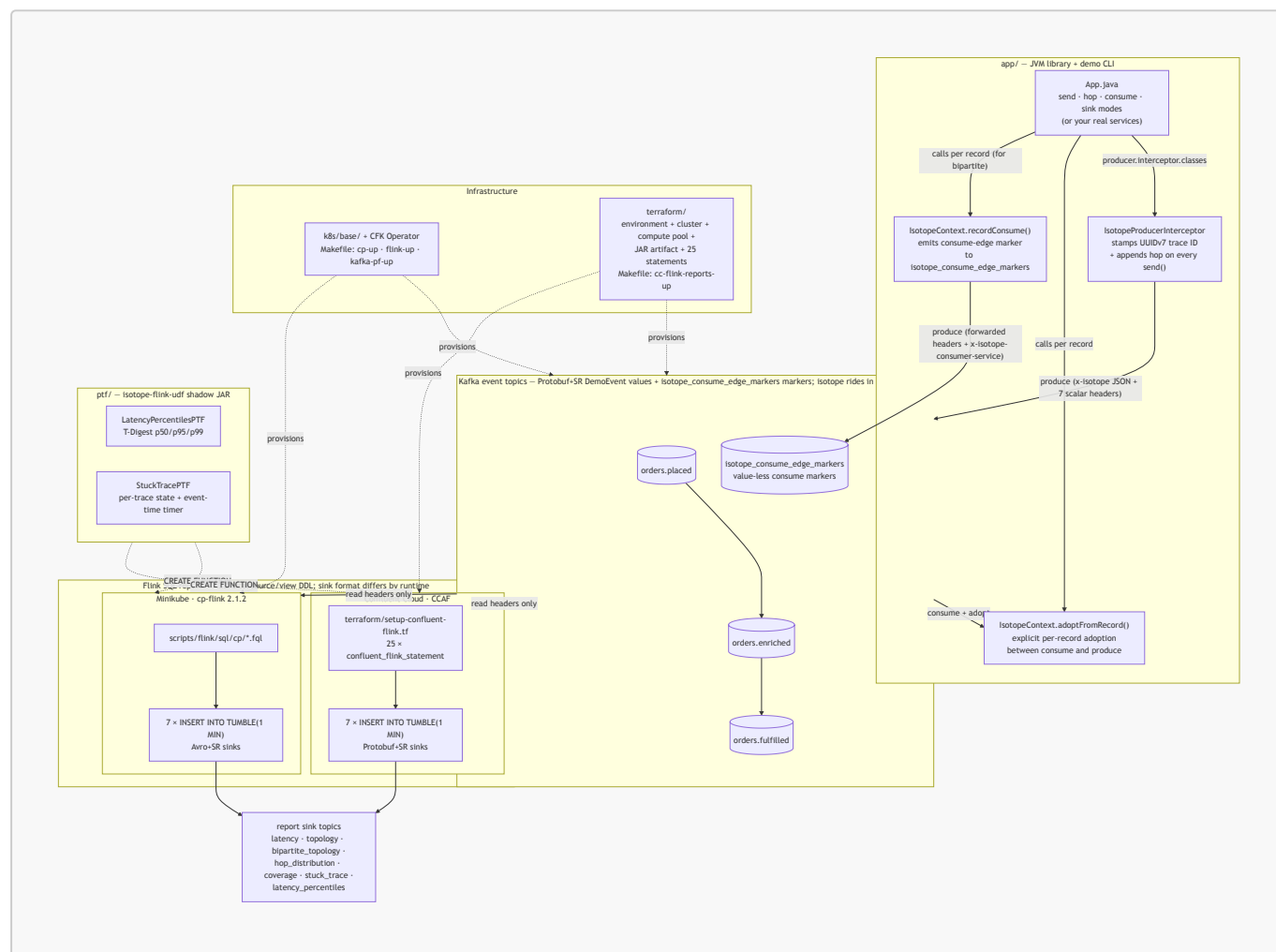
1.4 Why "bipartite"?

Background — why "bipartite"? The resulting topology report is an example of a **bipartite graph** from graph theory (a sub-field of discrete mathematics). A graph is *bipartite* when its vertices partition into two disjoint sets such that every edge connects a vertex in one set to a vertex in the other — edges never run within a set. Here the two sets are **services** (**svc-A**, **svc-B**, ...) and **topics** (**orders.placed**, **orders.enriched**, **orders.fulfilled**, **isotope_consume_edge_markers**); every edge is either a **produce edge** (**service → topic**, from the isotope's **hops[]**) or a **consume edge** (**topic → service**, from the **isotope_consume_edge_markers** markers). Kafka's pub/sub model guarantees the bipartite shape: service s never connect directly to other services, and topics never connect directly to other topics — every interaction flows through the opposite set. Before consume-side markers existed, only produce edges were captured, so the topology view collapsed into a **unipartite service → service → service** chain with topics hidden as edge labels and terminal consumers omitted entirely. With both edge directions now wired, topics become first-class nodes alongside services, and the **bipartite_topology** report renders the full graph — every produce *and* consume edge, in both directions.

2.0 Architecture

A bird's-eye view of the moving parts. The JVM library in [app/](#) registers a Kafka producer interceptor that stamps the isotope into record headers on every **send()**; consume-then-produce services adopt the inbound trace via an explicit **IsotopeContext.adoptFromRecord(record)** call; records flow through a 3-topic chain; Flink SQL reads only the headers and emits 1-minute aggregate reports. The same source/view DDL deploys to both runtimes — **CP** on Minikube applies **.fql** files under [scripts/flink/sql/cp/](#), and **CCAF** in Confluent Cloud applies inline **confluent_flink_statement** resources under [terraform/](#).

The shadow JAR from [ptf/](#) (which powers two of the seven reports) registers identically on both. (Kafka is drawn once below for brevity — each runtime provisions its own cluster.)



See § 3.0 for the file tree behind each box, and § 4.0 for the run commands.

3.0 Repo layout

app/	isotope JVM library + demo CLI +
tests	
src/main/proto/ai/signalroom/kafka/isotope/proto/	
demo_event.proto	DemoEvent message (Protobuf value
schema)	
src/main/java/ai/signalroom/kafka/isotope/	
Isotope.java	POJO + JSON codec + Hop +
fromHeaders()	
	+ UUIDv7 helpers (uuidV7Bytes /
uuidV7String)	
IsotopeContext.java	ThreadLocal + adoptFromRecord() +
	recordConsume() (emits consume-edge
markers)	
IsotopeProducerInterceptor.java	stamps/appends x-isotope + 7 scalar
	reporting headers on send()
IsotopeMetrics.java	optional Micrometer/Prometheus
exporter	
	for the 3 stateless reports (§ 4.6)
App.java	demo CLI – pipeline-position verbs

```

generic
    src/test/java/.../
    IsotopeContextRecordConsumeTest

    src/integrationTest/java/.../

    BipartiteTopologyIT,

    tests; produce/consume

forwarded)
    ptf/
    reports)
        src/main/java/ai/signalroom/kafka/isotope/flink/
        LatencyPercentilesPTF.java
state + timers)
    StuckTracePTF.java
    TDigests.java
    src/test/java/.../
k8s/base/
    confluent-platform-c3++.yaml
Control Center
    flink-basic-deployment.yaml
    flink-rbac.yaml
k8s/monitoring/
`make metrics-up`
    00-namespaces.yaml
    10-prometheus.yaml
stages

host.minikube.internal:9410/9411/9412
    20-grafana.yaml
datasource

produce/consume meters
    kustomization.yaml
    README.md
scripts/
    port-forward-kafka.sh
localhost:8081 → SR
    port-forward-taskmanager.sh
    deploy-cp-flink-reports.sh
sql/cp/*.fql to

    deploy-cc-flink-reports.sh
apply`

    cc-cli-env.sh
`terraform output`,

BOOTSTRAP /

```

(place / enrich / fulfill / ship) +

send / hop / consume / sink modes

IsotopeCodecTest,

(no broker needed)

BrokerSmokeIT, ProducerInterceptorIT, ThreeStageHopPropagationIT,

IsotopeTestHarness – live-broker

DemoEvent via SR-framed Protobuf (need Minikube CP + SR port–

Flink PTF shadow JAR (powers 2 of 7

T-Digest p50/p95/p99 (PTF: per-window

per-trace state + event-time timer

shared T-Digest (de)serialization

TDigestsTest

CFK manifests

Kafka / SR / Connect / ksqlDB /

cp-flink session cluster + CMF

RBAC for the cp-flink operator

optional metrics showcase (§ 4.6.5) –

dedicated 'monitoring' namespace

Prometheus pod/Service; scrapes host

via

Grafana pod/Service; auto-provisioned

+ 8-panel dashboard for the 6

`kubectl apply -k k8s/monitoring`

runbook + troubleshooting

localhost:30092 → Kafka,

Flink TaskManager web UI forward

builds shadow JAR + applies

the cp-flink session cluster

builds shadow JAR + wraps `terraform

for the CCAF path

pulls Kafka + SR creds from

builds the JAAS string, exports

SR_URL / KAFKA_KEY / KAFKA_SECRET /


```

JAAS / ...
  cc-app-run.sh
:app:run` that

six -D flags
  flink/README.md
(CP=7 reports/Avro+SR,

operations
  flink/sql/cp/
01_register_functions,

06_consume_events_view,

reports, 99_teardown

terraform/setup-confluent-flink.tf.)
terraform/
cc-flink-reports-up`)
  providers.tf
vars
  versions.tf
provider versions
  variables.tf
region, day_count
  data.tf
sources
  setup-confluent-environment.tf
governance package)
  setup-confluent-kafka.tf
rotation module

tf_module)
  setup-confluent-flink.tf
compute pool,

and 25 inline

resources: 4 ALTER TABLE

DROP FUNCTION +

INSERT INTO
  outputs.tf
rotating

(sensitive)
  terraform.png
§ 4.5)
Makefile
flink-reports-up /

reports-down /

```

```

thin wrapper around `./gradlew

sources cc-cli-env.sh and injects the

Flink SQL reports – runtime split

CCAF=7 reports/Protobuf+SR), layout,

CP Flink SQL: 00_source_table,

05_isotope_view,

05_report_sinks (avro-confluent),
10/20/25/30/40/60/70 INSERT INTO

(CCAF SQL is inlined under

CCAF infrastructure-as-code (`make

Confluent provider – cloud key/secret

required Terraform (>= 1.13) +

confluent_api_key/secret, cloud,

organization lookup + other data

environment (ESSENTIALS stream-

Kafka cluster + Kafka API key

(iac-confluent-api_key_rotation-

service account + 6 role bindings,

artifact upload, SR API key rotation,

`confluent_flink_statement`

+ 3 VIEW + 7 sink CREATE TABLE + 2

2 CREATE FUNCTION (both PTFs) + 7

environment_id, bootstrap, SR URL,

Kafka + SR API key/secret outputs

rendered resource graph (embedded in

cp-up / flink-up / kafka-pf-up /

cc-flink-reports-up / cc-flink-

```

metrics-up / metrics-down / metrics-

delete / ...

4.0 Running

Cheapest-first order if you're new: `./gradlew test` (§ 4.1) → local CP via Minikube (§ 4.2–4.4) → CCAF in the cloud (§ 4.5). Skip ahead if you only care about one runtime.

Just want the commands? The full local stack-up sequence (cluster → Kafka → Flink → reports → traffic → teardown) is consolidated in [docs/runbook-minikube.md](#).

4.1. Unit tests (no broker, instant)

```
./gradlew test                # both subprojects
# or scoped:
./gradlew :app:test           # IsotopeCodecTest (10) +
IsotopeContextRecordConsumeTest (4) – JSON roundtrip, hop eviction, UUIDv7
properties, consume-marker emission (14 tests)
./gradlew :ptf:test           # TDigestsTest – T-Digest sketch
(de)serialization + accuracy
```

4.2 Demo CLI — see one trace propagate live

The fastest way to watch the isotope mechanic. Requires the cluster to be up and the Kafka + SR forwards running (see step 3 below for the bring-up commands). The CLI has two argument styles — **pipeline-position verbs** that bake in the orders.* topic chain (recommended for the demo) and **generic verbs** that take raw topic + service args (for ad-hoc inspection on any topic):

Verb	Args	What it does
place	[payload]	Produces one isotope-tagged <code>DemoEvent</code> to <code>orders.placed</code> as <code>order-intake-service</code> (default payload: <code>hello</code>), then exits. Auto-creates the topic.
enrich	—	Consumes from <code>orders.placed</code> , adopts the isotope, emits a consume-edge marker to <code>isotope_consume_edge_markers</code> as <code>order-enrichment-service</code> , then re-produces to <code>orders.enriched</code> . Runs until Ctrl-C.
fulfill	—	Same as <code>enrich</code> but for <code>orders.enriched</code> → <code>orders.fulfilled</code> as <code>order-fulfillment-service</code> . Runs until Ctrl-C.
ship	—	Terminal consumer for <code>orders.fulfilled</code> as <code>shipping-notification-service</code> . Emits a consume-edge marker so it shows up in the bipartite report; does not re-produce. Runs until Ctrl-C.
send	<topic> <service> <payload>	Generic produce. Auto-creates the topic.

Verb	Args	What it does
hop	<in-topic> <out-topic> <service>	Generic consume-then-produce; emits a consume-edge marker. Runs until Ctrl-C.
consume	<topic> <service>	Generic terminal-consume; emits a consume-edge marker and pretty-prints the trail. Runs until Ctrl-C.
sink	<topic>	Passive peek — pretty-prints the isotope trail but does NOT emit a consume marker. Use for ad-hoc inspection. Runs until Ctrl-C.

4.2.1 Full Bipartite Demo

A 4-stage chain (full bipartite graph) in four terminals. Run them in pipeline order — **place** produces, then **enrich** / **fulfill** / **ship** each pick up where the previous stage left off:

```
# Terminal A — kick the chain off (run repeatedly to send more)
./gradlew :app:run --args="place 'hello world'" -q

# Terminal B — first hop: orders.placed → orders.enriched
./gradlew :app:run --args="enrich" -q

# Terminal C — middle hop: orders.enriched → orders.fulfilled
./gradlew :app:run --args="fulfill" -q

# Terminal D — terminal consumer (prints the full 3-hop trail AND emits a
#                  consume-edge marker so shipping-notification-service shows up
#                  in the bipartite report)
./gradlew :app:run --args="ship" -q
```

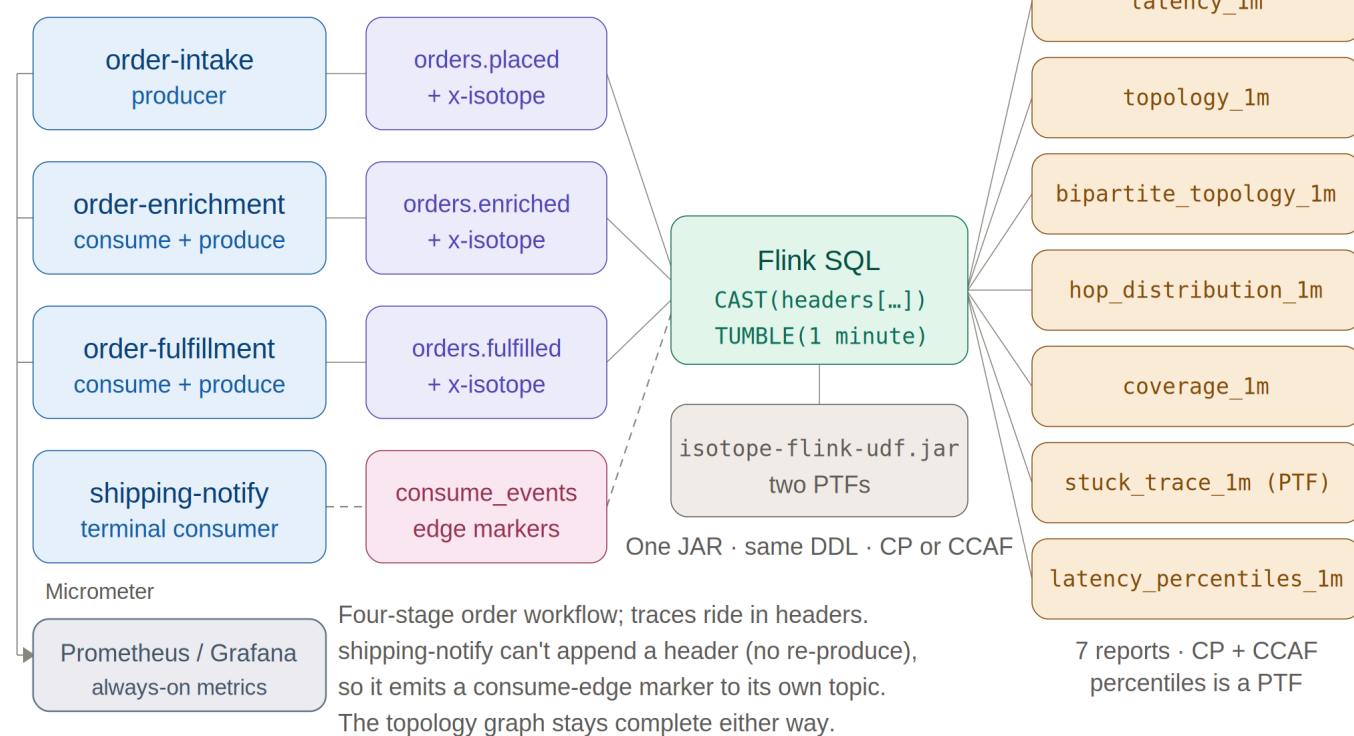
Terminal D's output for each record shows the same **trace_id** across all three hops, **origin** = **order-intake-service** (never reassigned), and **hops[]** listing **order-intake-service** → **order-enrichment-service** → **order-fulfillment-service** in order with per-hop timestamps. The bipartite-topology report sees all six edges: produce edges **order-intake-service** → **orders.placed**, **order-enrichment-service** → **orders.enriched**, **order-fulfillment-service** → **orders.fulfilled** and consume edges **orders.placed** → **order-enrichment-service**, **orders.enriched** → **order-fulfillment-service**, **orders.fulfilled** → **shipping-notification-service**. Swap **consume** for **sink** if you only want to inspect records without recording the terminal edge. Override endpoints via **-Dkafka.bootstrap=...** / **-Dschema.registry.url=...** if you're not on the default Minikube layout.

Collector

Stamp headers; emit markers + metrics

Interpreter

Reads headers + markers



- | | | |
|--|--|---------------------------------------|
| ☐ Service | ☐ Event topic (x-isotope) | ☐ Marker topic |
| ☐ Flink interpreter | ☐ Report sink (1-min) | ☐ Shadow JAR |
| ☐ Metrics backend | — Produce | --- Edge marker emit |

4.3 Integration tests (live Kafka via Minikube)

Bring up the local Confluent Platform stack and port-forward Kafka + SR:

```
make minikube-start          # one-time
make cp-up                   # CFK Operator +
Kafka/SR/Connect/ksqlDB/C3 (~5 min)
make kafka-pf-up             # localhost:30092 → Kafka,
localhost:8081 → Schema Registry
```

Then run the suite:

```
./gradlew :app:integrationTest #
every IT below
./gradlew :app:integrationTest --tests '*ProducerInterceptorIT' #
just one
```

Override the endpoints if needed:

```
./gradlew :app:integrationTest \
  -PkafkaBootstrap=localhost:30092 \
  -PschemaRegistryUrl=http://localhost:8081
```

Tear down forwards when done:

```
make kafka-pf-down
```

The integration tests cover:

Test	What it verifies
<code>BrokerSmokeIT</code>	AdminClient can create/list/delete a topic via the NodePort port-forward
<code>ProducerInterceptorIT</code>	A consumer sees the <code>x-isotope</code> JSON header + all 7 scalar reporting headers with the expected origin/hop values, and the Protobuf round-trip preserves <code>DemoEvent.source / payload</code>
<code>ThreeStageHopPropagationIT</code>	<code>order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service</code> produces a stable trace ID, 2-hop trail in send order, and correct scalar headers (origin = <code>order-intake-service</code> , this = <code>order-enrichment-service</code> , hop count = 2) at the terminal; consume-then-produce hops use <code>IsotopeContext.adoptFromRecord</code> to carry the trace forward
<code>BipartiteTopologyIT</code>	The 4-stage <code>order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service → topic-CD → shipping-notification-service</code> chain emits exactly three consume-edge markers to a per-test markers topic — one per consume edge. Every marker carries the trace ID, forwarded <code>x-isotope*</code> scalars describing the upstream producer, and the new <code>x-isotope-consumer-service</code> naming the downstream consumer. Asserts the (<code>consumer_service</code> , <code>consumed_topic</code>) set is exactly the three pairs of stages 2-4

4.4 Flink SQL reports on Confluent Platform for Apache Flink (Minikube)

Seven reports — five pure Flink SQL plus two JAR-backed PTFs — run against a `cp-flink` session cluster (Flink 2.1.2) managed by the Confluent Flink Kubernetes Operator. The same FQL files deploy to Confluent Cloud — see § 4.5 for that path; this section is the local-Minikube one.

The full bring-up sequence — cluster → Flink → reports → traffic → teardown — is consolidated in [docs/runbook-minikube.md](#). The short version: `make flink-up` then `make flink-reports-up`, then drive traffic across **multiple** 1-minute windows (a single burst sits in one open window forever — the watermark has to cross `window_end` for a tumbling window to emit) and wait ~90s after the last record.

Report sink topics ride **Avro+SR** (`avro-confluent`, auto-registered on first write) so Control Center renders them natively — a deliberate *format-by-domain* split: app events are **Protobuf+SR** (`DemoEvent`), Flink aggregates are **Avro+SR** (cp-flink ships no SR-integrated Protobuf format), and the consume-edge marker topic `isotope_consume_edge_markers` is **null-value / headers-only**. Not a defect — a clean split by domain.

4.5 Flink SQL reports on Confluent Cloud for Apache Flink (CCAF)

The CCAF parallel of § 4.4, driven by Terraform under `terraform/` — no local cluster. One `make` target spins up a fresh Confluent Cloud environment, Kafka cluster, compute pool, two rotating service-account API key pairs (Kafka + Schema Registry), the uploaded PTF JAR, and 25 `confluent_flink_statement` resources (4 ALTER TABLE + 3 VIEW + 7 sink CREATE TABLE + 2 CREATE FUNCTION + 7 INSERT INTO, plus 2 transient DROP FUNCTION). The shape mirrors `apache_flink-kickstarter-ii` — same provider version and `iac-confluent-api_key_rotation-tf_module`.



The full provision → deploy → traffic → teardown sequence is consolidated in `docs/runbook-ccaf.md`.

The short version: `export` a Cloud-level Confluent Cloud API key, `make cc-flink-reports-up` (~6–8 min first run; idempotent re-applies), drive traffic with `scripts/cc-app-run.sh place|enrich|fulfill|ship` across **multiple** 1-minute windows (a single burst sits in one open window forever, and you wait ~90s after the last record), then `make cc-flink-reports-down` to `terraform destroy` the whole environment.

Two CCAF-specific design points worth keeping in the README:

Format-by-runtime (not -by-domain). CP's reports land on **Avro+SR** (`'value.format' = 'avro-confluent'` in `scripts/flink/sql/cp/05_report_sinks.fql`). CCAF's reports land on **Protobuf+SR** (`'value.format' = 'proto-registry'` in each sink's WITH clause in `terraform/setup-confluent-flink.tf`). The two runtimes' SQL is otherwise unshared: CP's lives hardcoded in `scripts/flink/sql/cp/`, CCAF's lives inline as `confluent_flink_statement` resources in `terraform/setup-confluent-flink.tf`.

Why percentiles is a PTF. CCAF rejects all `CREATE FUNCTION` statements for user-defined *aggregate* functions ("aggregate functions are not supported"). Percentiles would naturally be an aggregate, so to keep the report portable it's implemented as a `ProcessTableFunction` instead —

`LATENCY_PERCENTILES` (class `LatencyPercentilesPTF`) does its own 1-minute tumbling-window aggregation over a T-Digest sketch via per-window state and event-time timers. A PTF has no such restriction, so it registers and runs on **both** runtimes, exactly like `STUCK_TRACE_PTF`. Both runtimes therefore run the same seven reports: `latency` (avg/min/max), `topology` (produce-side), `bipartite_topology` (full service↔topic↔service graph), `hop_distribution`, `coverage`, `stuck_trace`, and `latency_percentiles` (p50/p95/p99).

4.6 Stateless reports via Micrometer → Prometheus/Grafana (optional)

Is Flink overkill for these reports? **Yes and no** — it's not all about stateless aggregation. Of the seven reports, **three are pure stateless scalar aggregation** keyed on bounded-cardinality dimensions (service / topic / hop_count — never `trace_id`):

- `latency_1m` (`10_latency_report.fql`) — avg/max latency per (`pipeline`, `origin_service`, `this_topic`).
- `topology_1m` (`20_topology_report.fql`) — produce-edge record counts.

- **hop_distribution_1m** (`30_hop_distribution.fql`) — records per **hop_count**.

These don't need a stream processor. The **producer interceptor** already has every value in scope on each `send()`, so it can emit them as **Micrometer** meters and let **Prometheus** do the 1-minute windowing at query time (`rate()` / `increase()`), with **Grafana** on top. The other four reports — **latency_percentiles** (mergeable T-Digest), **coverage**, **bipartite_topology**, **stuck_trace** — are per-**trace_id** *stateful* or *absence-of-event* problems Prometheus can't express, so they **stay in Flink**. This path is **additive and opt-in** (off by default); it doesn't replace the Flink reports — it's the cheaper way to serve the three that are metrics, not stream processing.

Emission lives in one place — **IsotopeMetrics.java** — mirroring how **TDigests** centralizes the sketch contract.

4.6.1 Enabling the exporter

Pass `-Dmetrics.prometheus.enabled=true` to any long-running stage. **Producing** stages (**hop** / **enrich** / **fulfill**) emit the produce-side meters via the interceptor; **consuming** stages emit the consume-side meters via the marker path (§4.6.3) — **hop** does both (it consumes *and* produces), and the terminal **consume** / **ship** stages emit the consume side alone. The app serves the meters at `GET /metrics` on `metrics.prometheus.port` (default **9404**) via the JDK's built-in HTTP server — no extra runtime.

```
# Prereq: 'make kafka-pf-up' is up. Run the enrich stage with metrics
exposed:
./gradlew :app:run -Dmetrics.prometheus.enabled=true --args="enrich"
# → metrics on http://localhost:9404/metrics

# In another shell, drive traffic so the meters move (origin produce):
for i in {1..30}; do ./gradlew :app:run --args="place burst-$i" -q; sleep 5;
done

# Scrape:
curl -s localhost:9404/metrics | grep isotope_
```

Point a Prometheus scrape job at each stage's **:9404** and the three reports become PromQL queries (no report topics, no Control Center — Grafana instead). Unlike the Flink jobs, there's **no watermark wait**: cumulative counters update on every send and Prometheus windows them at read time.

Reading the latency metric — mind the backlog. **hop** / **consume** use a *random* consumer group with `auto.offset.reset=earliest`, so a fresh stage **replays the whole topic from offset 0**. Latency is `now - origin_ts`, so days-old replayed records report enormous values — e.g. an `isotope_hop_latency_seconds_sum` of ~9,000,000 over 58 records (~43 h average) is just the backlog, not steady-state latency (the Flink `latency_1m` report shows the same, by the same definition). The tell: `_count` exceeds the number of records you just sent. Two ways to see realistic latency: drain the backlog and read `rate(isotope_hop_latency_seconds_sum[1m]) / rate(..._count[1m])` (the windowed rate ignores the stale backlog once it stops growing), or skip the backlog entirely with `-Disotope.consume.from=latest` so the stage only consumes records produced after it starts:

```
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Disotope.consume.from=latest --args="enrich"
```

4.6.2 The three reports as PromQL

Two meters cover all three reports — a **Timer** whose count doubles as the topology edge count, plus a **Counter** for the hop histogram:

Report	Meter (Prometheus name)	Labels
latency_1m + topology_1m	isotope_hop_latency_seconds_{count,sum,max} (Timer)	pipeline, origin_service, this_service, this_topic
hop_distribution_1m	isotope_hop_records_total (Counter)	pipeline, this_topic, hop_count

```
# latency_1m – avg latency (seconds)
sum by (pipeline, origin_service, this_topic)
(rate(isotope_hop_latency_seconds_sum[1m]))
/ sum by (pipeline, origin_service, this_topic)
(rate(isotope_hop_latency_seconds_count[1m]))

# latency_1m – max
max by (pipeline, origin_service, this_topic)
(isotope_hop_latency_seconds_max)

# topology_1m – records per produce edge
sum by (pipeline, origin_service, this_service, this_topic)
(increase(isotope_hop_latency_seconds_count[1m]))

# hop_distribution_1m – records per hop_count
sum by (pipeline, this_topic, hop_count)
(increase(isotope_hop_records_total[1m]))
```

4.6.3 Consume-side meters

The three meters above all come from the **produce** side — the interceptor on `send()`. The **consume** side adds three more. Two — the edge counter and the time-to-consume latency — are emitted by `IsotopeContext.recordConsume` right beside the consume-edge marker it writes to `isotope_consume_edge_markers`. The third, `isotope.consume.age`, is emitted **once per consumed record on whichever path the consumer takes**: continuing consumers report it from the **adoption path** (`IsotopeContext.adoptFromRecord(record, service)`), and terminal consumers — which never adopt — report it from the **marker path** (`recordConsume`, guarded on `current() == null` so a stage that does both, like `hop`, never double-counts). So age fires on every consuming stage: `hop` via adoption, the terminal `consume` / `ship` stages via the marker. Same gating — no-op unless the exporter is started.

Signal	Meter (Prometheus name)	Labels
consume-edge counts (topic→consumer)	<code>isotope_consume_records_total</code> (Counter)	pipeline, this_topic, consumer_service
time-to-consume latency (origin→consume)	<code>isotope_consume_latency_seconds_{count,sum,max}</code> (Timer)	pipeline, origin_service, consumer_service, this_topic
origin→adoption age (how stale at pickup)	<code>isotope_consume_age_seconds_{count,sum,max}</code> (Timer)	pipeline, origin_service, consumer_service, this_topic

```
# consume edges – records consumed per (topic → consumer) per minute
sum by (pipeline, this_topic, consumer_service)
(increase(isotope_consume_records_total[1m]))

# time-to-consume – avg seconds from trace origin to consumption
sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_latency_seconds_sum[1m]))
/ sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_latency_seconds_count[1m]))

# consume age – avg seconds a record had aged by the time a service adopted
it
sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_age_seconds_sum[1m]))
/ sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_age_seconds_count[1m]))
```

These are **net-new signals**, not a port of a Flink report. `isotope_consume_records_total` is the *consume half* of the bipartite topology — the topic→consumer edge tallies — and `isotope_consume_latency_seconds_*` is an origin→consume figure no Flink report computes (the Flink `latency_1m` measures origin→**produce**-hop). The **full bipartite_topology** report still stays in Flink: stitching produce and consume edges per `trace_id` into one graph is per-trace stateful, which a counter can't do. What you get here is the bounded-cardinality edge *counts* and a time-to-consume distribution — cheap, windowable at read time, and free of the watermark wait.

`isotope_consume_age_seconds_*` measures the **same now – origin_ts** quantity as `isotope_consume_latency_seconds_*`, but it's the **universal consume-side age signal** — emitted once on every consuming stage, where latency only fires on the marker path. That's the point of having both: query `isotope_consume_age_*` to get consume-lag across **all** consumers (including **hop**-style stages that adopt-and-forward without ever writing a marker), and the timer's `_count` doubles as the per-`(consumer_service, topic)` consume rate. At a terminal consumer the age equals that consumer's latency by definition (same **now – origin_ts**); the two diverge in *coverage*, not value — latency is marker-only, age is everywhere.

The same **backlog caveat** from §4.6.1 applies: `hop` / `consume` / `ship` use a random consumer group with `auto.offset.reset=earliest`, so a fresh stage replays from offset 0 and both `isotope_consume_latency_seconds_sum` and `isotope_consume_age_seconds_sum` then reflect days-old origins, not steady-state. Read the windowed `rate(...sum[1m]) / rate(...count[1m])`, or start with `-Disotope.consume.from=latest`.

4.6.4 What stays in Flink — and two deliberate gaps

Moving these out isn't free — two columns the Flink reports carry have **no Prometheus equivalent**, and both are documented in `IsotopeMetrics.java`:

- **No `distinct_traces`**. All three SQL reports carry a `COUNT(DISTINCT trace_id)` column. A counter can't dedup, and `trace_id` is unbounded-cardinality so it can't be a label. If you need it, that column stays in Flink (or approximate it with a HyperLogLog sketch).
- **No windowed `min` latency**. A Micrometer `Timer` exposes max but not a per-window min — avg and max port cleanly, min does not.

So the line between "metric" and "Flink job" runs *through* a couple of these reports, not cleanly between them — which is exactly why the move is opt-in rather than a wholesale replacement.

Why `latency_percentiles_1m` is NOT in this list. Percentiles *can* be served by Prometheus — but only via a *classic* histogram (`publishPercentileHistogram()` + `histogram_quantile()`), whose accuracy is **bucket-bound, not adaptive**: error is the width of fixed, pre-chosen buckets, so the tail (p99) is only as good as your bucket layout, and covering a range finely means emitting hundreds of `le` series per tag combo. Prometheus's adaptive answer — *native histograms* (exponential buckets, the closest thing to a `T-Digest`) — isn't emittable through Micrometer yet (experimental, protobuf-only as of late 2024). So the `T-Digest PTF` wins on tail accuracy **and** scales better: its sketch is bounded (~few KB/key) and mergeable, whereas at production volume a Micrometer percentile-histogram's `le`-bucket cardinality grows with the range you need to resolve. The built-in Flink `PERCENTILE` aggregate (exact, pure SQL) is *also* the wrong call at high volume — it retains every value in the window — so percentiles stay a **T-Digest sketch PTF** on purpose (see that file's header for the full rationale). This is a 3-Micrometer / 4-Flink split, not 4/3.

4.6.5 One-command showcase: Prometheus + Grafana on Minikube

To see these meters instead of `curl`-ing `/metrics`, there's an optional, self-contained stack under `k8s/monitoring/` — Prometheus + Grafana as Minikube pods, with the datasource and a dashboard (all six produce/consume signals from §4.6.2–4.6.3) **auto-provisioned**, so it opens straight to a populated board with no login.

The pipeline stages run on your **host** via `./gradlew :app:run`, not in-cluster — so Prometheus scrapes back across the Minikube→host bridge `host.minikube.internal`, one host port per stage (`enrich`→9410, `fulfill`→9411, `ship`→9412; edit `k8s/monitoring/10-prometheus.yaml` to change the mapping).

```
make metrics-up          # deploy, wait, port-forward Prometheus+Grafana, open
                           Grafana

# In separate terminals, run each stage on its mapped port (metrics on):
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9410 \
```

```
-Disotope.consume.from=latest --args="enrich"      # fulfill→9411, ship→9412  
# ...then drive traffic with `place` so the meters move.  
  
make metrics-down      # stop the port-forwards (pods stay up)  
make metrics-delete    # tear the whole showcase down
```

Confirm scraping at Prometheus → [Status](#) → [Targets](#) (a target is **DOWN** until you start its stage — expected).
Full runbook and troubleshooting: [k8s/monitoring/README.md](#).

5.0 Resources

- [Medium Article: Kafka's quiet observability superpower — Kafka Interceptors](#)